Data bases 2

Indice

Transaction manager

The transaction manager is one of the modules that make up a DBMS. The system handles, as the name implies, transactions, **elementary/atomic units of work performed by applications**. Each transaction is conceptually encapsulated by the `begin transaction` and `end transaction` commands. Within one, one of `commit-work` or `rollback-work` is executed to respectively commit the transaction or abort it. Each transaction obeys the ACID properties.

Concurrency control

Manages the simultaneous execution of transactions and avoids the insurgence of anomalies. The goal of this controller is to achieve the maximum number of transaction per second (TPS). Two concurrent transaction may be executed *interleaved* or *nested*.

We can encounter the following anomalies:

1. **Lost update**: one of the updates is overwritten by the other; It occurs when both transactions read the same data and then apply the update. Pattern: `r1 - r2 - w2 - w1`
2. **Dirty read**: one of the transactions reads data modified by a transaction that will abort. Pattern: `r1 - w1 - r2 - abort1 - w2`
3. **Non-repeatable read**: another transaction interferes with the data used by a transaction. Pattern: `r1 - r2 - w2 - r1`
4. **Phantom update**: a constraint is broken because another transaction interfered with the data. Pattern: `r1 - r2 - w2 - r1`
5. **Phantom insert**: one transaction inserts new data that may change the values used in a different transaction. Pattern: `r1 - w2(new) - r1`

Concurrency theory builds upon a model of transaction and concurrency control that helps understanding real systems. Commercial systems exploit implementation level mechanisms which help achieve some of the desirable properties postulated by theory.

Scheduler

An operation is a read/write of a specific datum by a specific transaction. A schedule is sequence of operations performed by concurrent transactions that respects the order of operations of each transaction. For a 2 transactions we have 6 different orderings: 2 serial, 2 interleaved and 2 nested.

Our goal is to reject schedules that cause anomalies. A scheduler is the component that accepts or rejects the operations requested by transactions. The only schedule that does not cause anomalies it the serial one: one in which the action of each transaction occur continuously. If we have an interleaved/nested one, we need to check that it causes the same modifications as a serial one: **a serializable schedule is a schedule that leaves the database in the same state as some serial schedule**.

At first we will consider with the following assumptions:

1. The transactions always succeeds (no aborts)

2. The transactions are observed a-posteriori

The cardinality of the set of serial schedules of n transactions of k_i is $N_s = n!$, however the cardinality of the total number of possible schedules is $N_d = \frac{(\sum_i^n k_i)!}{\prod_i^n (k_i!)}$. We have that $N_s \ll N_d$.

View serializability

1. $r_i(x)$ **reads from** $w_j(x)$ **in a schedule when** $w_j(x)$ **precedes** $r_i(x)$
2. $w_i(x)$ **is a final write if it is the last write on** x **that occurs in a schedule**

Two schedules are **view-equivalent if they have the same operations, the same reads-from relationships, the same final writes**. A schedule is **view-serializable if it is view-equivalent to serial one on the same transactions**. The class of view-serializable schedules is named VSR.

Determining the view equivalence of a schedule is done in polynomial time. However we need to try all possible serial schedules, which are $n!$, thus VSR is a **NP-complete problem**.

Conflict serializability

To have a faster algorithm, we need to restrict VSR. Two definitions:

1. **Two operations** o_i **and** o_j **(**$i \neq j$**) are in conflict if they address the same resource and at least one of them is a write**
2. Two schedules are **conflict equivalent (**$\approx_C$**) if both contain the same operations and in all the conflicting pairs the transactions occur in the same order**

A schedule is **conflict-serializable if and only if it is conflict-equivalent to a serial schedule of the same transactions**. The class of conflict-serializable schedules is named CSR. We have that $CSR \subset VSR$, so then we have $CSR \implies VSR$.

We can test CSR with the **conflict graph**:

- One **node for each transition** T_i
- One **arc from** T_i **to** T_j **if there exists at least one conflict between an operation** o_i **of** T_i **and an operation** o_j **of** T_j **such that** o_i **precedes** o_j.

A schedule is in CSR if and only if its conflict graph is acyclic.

Concurrency control in practice

CSR can be useful only if we know the conflict graph in advance. However this cannot happen irl. A scheduler must be capable of working online. We need to remove the "a posteriori" hypothesis.

Until now we have been working with an "a posteriori" view of the execution. When working with online schedulers, we need to consider **arrival sequences, i.e sequences of operations requests emitted by transactions**. We will abuse notation and refer to arrival sequences in the same way as "a posteriori" views.

We have two approaches to online scheduling:

1. **Pessimistic, based on locks**
2. **Optimistic, based on timestamps**

Locking The most common method. A transaction is well formed if:

- reads are preceded with a `r_lock` (**shared lock**) and followed by **unlock**
- writes are preceded with a `w_lock` (**exclusive lock**) and followed by **unlock**

Transactions that first read and then write may acquire a `w_lock` when reading or acquire a `r_lock` first and the upgrade it to a `w_lock` (*lock escalation*).

An object can be in 3 different states: **free, r-locked, w-locked**.

The lock manager receives the primitives from the transactions and grants access to the resources according to the conflict table:

Request	Free	r-locked	w-locked
`r_lock`	**OK** -> r-locked	**OK** -> r-locked++	KO -> w-locked
`w_lock`	**OK** -> w-locked	NO -> r_locked	KO -> w-locked
`unlock`	ERROR	**OK** -> lock–	**OK** -> free

Each r-lock has a counter `n` that counts the number of concurrent readers.

When access to a resource is not granted, the transaction is put in a waiting queue until the resource unlocks.

Locks are implemented with **lock tables**, which are basically **hash tables** indexed by the hash of the resource locked. **Each locked item has a linked list associated with it**. Every node in the list represents the transaction which requested the lock, the lock mode and the current status (granted/waiting).

Respecting locks, however, is not enough to grant serializability. To achieve serializability, we **need to ensure the two-phase rule: a transaction needs to first acquire all the needed locks and then it can start releasing them**. The schedules generated by this methods are in the **2PL** set and we have that $2PL \subset CSR \subset VSR$.

2PL can deal with most anomalies: non-repeatable read, lost update, phantom update, phantom insert (needs the introduction of **predicate locks**, i.e locks on all data that matches a predicate). We still have problems with dirty reads and aborts. This is because we are still considering commit-projection as an hypothesis.

To remove the commit-projection hypothesis we need to add a constraint to 2PL, making it **strict 2PL: locks are held until commit/rollback**.

In commercial systems, different isolation levels are allowed. In SQL99 we have:

1. `READ UNCOMMITTED`: no locks, every anomaly
2. `READ COMMITTED`: it adds read locks, however without 2PL (prevents dirty reads)
3. `REPEATABLE READ`: 2PL for reads (prevents dirty reads, non-repeatable reads and phantom updates)
4. `SERIALIZABLE`: avoids all anomalies (2PL with predicate locks)

Serializable transactions don't execute serially! The requirement is that the end result should be the same as if they executed serially. Locking introduces synchronization problems:

1. **Deadlocks**: two or more transactions in endless mutual wait
2. **Starvation**: a single transaction in endless wait

Thus the `SERIALIZABLE` level is used sparingly.

Deadlocks Occurs because concurrent transactions hold and in turn request resources held by other transactions. To analyze deadlocks we can draw:

1. **Lock graphs**: specifies who holds what (nodes are resources or transactions and arcs are lock assignments/requests)
2. **Wait-for graph**: a simplification of lock graphs in which nodes are transactions and arcs are *waits for* relationships.

A deadlock is represented by a cycle in the wait-for graph of transactions.

We have various ways of resolving deadlocks:

1. **Timeout**: we kill transactions after a long wait
   - We cannot determine if a transaction is simply waiting for along time or is blocked
   - Choosing a proper timeout is difficult: too long is useless in case of deadlocks, too short leads to un-required kills. Timeout is usually variable and system decided.
2. **Deadlock prevention**: transactions killed when they could be in a deadlock. Preventions worked using heuristics

3. **Deadlock detection**: Transactions killed when they are in a deadlock. Detection works by analyzing the wait-for graph

Deadlock prevention We have two methods:

1. **Resource-based prevention**: restricts lock requests
   - Transactions requests resources all at once and only once
   - Resources are globally sorted and must be requested *in global order*
   - It is not easy for transactions to anticipate all requests
2. **Transaction-based prevention**: restrictions based on the transaction's ID
   - We assign IDs sequentially, making it possible to calculate a transaction's age
   - We can prevent older transactions waiting for younger ones to end:
     - **Non-preemptive (wait-die)**: if requesting transaction (`RT`) `T1` is older than conflicting transaction (`CT`) `T2`, then `T1` waits, otherwise it dies.
     - **Preemptive (wound-wait)**: if `RT T1` is older thhan `CT T2`, then `T1` is wounded, otherwise `T1` waits.
   - We can preemptively or non-preemptively choose the transaction to kill

Deadlock detection Requires an algorithm to detect cycles in the wait-for graph. It must work with distributed resources. We will use **Obermack's algorithm**.

Assumptions:

1. Transactions execute on a single main node
2. Transactions may be decomposed in *sub-transactions* running on other nodes
3. When a transaction spawns a sub-transaction, it suspends work until the latter completes (Synchronicity)
4. We have two wait-for relationships:
   - T_i waits for T_j on the same node because T_i needs data locked by T_j,
   - A sub transaction of T_1 waits for another sub-transaction of T_i running on a different node

Obermarck's algorithm works locally and needs to exchange minimal amount of data with other nodes. It does not need to keep the whole global view. **Each node has a projection of the global dependencies**. Nodes exchange information and update their local view. **Communication is optimized to avoid that multiple nodes detect the same deadlock**.

A node A sends its local info to B only if: A contains a transaction T_1 that is waited for from another remote transaction and waits for a transaction T_j active on B and $i > j$. **Mnemonically: A sends info if a distributed transaction listed at A waits for a distributed transaction listed at B with smaller index**.

Periodically:

1. **Get the graph information from "previous nodes"**
2. **Update the local graph by merging the received information**
3. **Check the existence of cycles** among transactions denoting potential deadlocks. **If one is found, select one of the transactions and kill it**.
4. **Send updated graph info to the "next nodes"**

There are **some arbitrary choices in the algorithm**: sending messages if $i < j$ or $i > j$, sending messages to the following or preceding node. **Therefore there are 4 variants of the algorithm. They are all equivalent**.

Limiting deadlocks **The deadlock probability is much less than that of a conflict**. Considering a file with n records and two transactions doing two accesses to their records the conflict is $\mathcal{O}(1/n)$ while deadlocks are $\mathcal{O}(1/n^2)$.

The probability of a deadlock is linear in the number of transactions, but quadratic in record size, thus shorter transactions are healthier.

We have techniques that can reduce the frequency of deadlocks.

Update locks The **most frequent type** of deadlock occurs when **2 concurrent transactions start by reading the same resource and then deciding to write and try to upgrade the locks. Update locks are another type of lock that grants a read and a successive write**. They are very easy to implement and mitigate the most common collision: `r1(x) - r2(x) - w1(x) - w2(x)`.

Request	Free	Shared	Update	Exclusive
Shared	**OK**	**OK**	**OK**	KO
Update	**OK**	**OK**	KO	KO
Exclusive	**OK**	KO	KO	KO

Update locks are requested by using `SELECT FOR UPDATE` SQL statement.

Hierarchical locking Hierarchical locking makes **locks have more granularity** than locking or not the entire table. The objective is **locking as less as possible** to increase concurrency. A possible hierarchy can be: table, page, tuple, value.

To implement hierarchical locking we need to **introduce new locks that express the intention of locking at higher/lower granularity**:

1. **ISL**: intention of locking a subelement in shared mode.
2. **IXL**: intention of locking a subelement in exclusive mode.
3. **SIXL**: lock the current element in shared mode with intention of locking a subelement in exclusive mode (union of SL and IXL).

With new locks, we have a new granting table:

Request	Free	ISL	IXL	SL	SIXL	XL
ISL	**OK**	**OK**	**OK**	**OK**	**OK**	KO
IXL	**OK**	**OK**	**OK**	KO	KO	KO
SL	**OK**	**OK**	KO	**OK**	KO	KO
SIXL	**OK**	**OK**	KO	KO	KO	KO
XL	**OK**	KO	KO	KO	KO	KO

Locks are **requested starting from the root and going down the hierarchy. Locks are released starting from the locked resource going up the hierarchy**.

- To **request an SL or ISL** locks on a non-root element, a transaction **must hold an equally or more restrictive lock (ISL or IXL) on its parent**.
- To **request an IXL, XL or SIXL** lock on a non-root element, a transaction **must hold an equally or more restrictive lock (SIXL or IXL) on its parent**.

Timestamps Concurrency control based on timestamps is a method **complementary to 2PL** that, instead of assuming that conflicts will occur, **assumes that conflicts are rare**. This means we **first run the transaction and then calidate the operation before commit or before each operation**.

A timestamp is an identifier that defines a total ordering of events in a system. Each transaction has a timestamp representing the time at which it begun. **A schedule is accepted only if it reflects the serial ordering of the transactions induced by their timestamps**.

The scheduler has **two counters** $RTM(x)$ **and** $WTM(x)$ **for each object**:

1. $RTM(x)$: the timestamp of the **transaction with the highest one** that has **read** x
2. $WTM(x)$: the timestamp of the **transaction with the highest one** that has **written** x

The **scheduler receives requests tagged with the timestamp of the requesting transaction**:

1. $r_{ts}(x)$
   - Is **rejected** if $ts < WTM(x)$ and the transaction is **killed**

- **Otherwise access is granted** and $RTM(x) = max(RTM(x), ts)$

2. $w_{ts}(x)$
   - Is **rejected** if $ts < RTM(x) \vee ts < WTM(x)$ and the transaction is **killed**
   - **Otherwise access is granted** and $WTM(x) = ts$

2PL and TS are incompatible: we can find schedules that are 2PL that are not TS and viceversa. However we have that $TS \implies CSR$.

Basic **TS-based control works with commit-projection**. If aborts occur, the problem of dirty reads can still occur. **To cope with it** a variant, similar to long duration locks, must be used: **a transaction that issues** $r_{ts}(x)$**such that** $ts > WTM(x)$ **has its operation delayed until the last transaction commits or aborts**.

TS with Thomas' Rule Thomas' rule can be used to reduce the number of transactions killed in TS. It is a variation of basic TS.

1. $r_{ts}(x)$
   - Is rejected if $ts < WTMx$ and the transaction is killed
   - Otherwise access is granted and $RTMx = max(RTM(x), ts)$
2. $w_{ts}(x)$
   - Is **rejected** if $ts < RTM(x)$ and the transaction is **killed**
   - **Else**, if $ts < WTM(x)$ our write is obsolete and can be **skipped**
   - **Otherwise access is granted** and $WTM(x) = ts$

The rationale behind the rule is that of skipping a write on an object that has already been written by a younger transaction without any killing.

This rule extends TS into a new set (TS') that is not directly contained in CSR nor in VSR.

Multiversion concurrency control A variation on TS is that **reads are always accepted and writes simply generate new versions and reads access only the right version**. Once no old versions are needed, they are discarded. **This means that there is a unique global** RTM **and multiple** WTM_i **for a single resource**.

1. $r_{ts}(x)$ **always succeeds**. A copy x_k is selected for reading such that:
   - If $ts \geq WTM_N(X)$ then $k = N$
   - Else tajke k such that $WTM_k(x) \leq ts < WTM_{k+1}(x)$
2. $w_{ts}(x)$
   - Is **rejected** if $ts < RTM(x)$
   - **Otherwise a new version is created** for timestamp ts. The number N of active versions is incremented. The **various versions are kept sorted from oldest to youngest**.

Like TS', the set of schedules serializable with **TS(multi) is not directly related to VSR nor CSR**.

Snapshot isolation Snapshot isolation is new isolation level that **works similarly to TS(multi): no RTM is used, only WTMs**. Every transaction **reads the version consistent with its timestamp and defers writes to the end**. If the **scheduler detects that the writes of a transaction conflict** with writes of other concurrent transactions after the snapshot timestamp, **it aborts**.

Snapshot isolation **does not guarantee serializability**. For example the following transactions

```
update Balls set Color="White" where Color="Black"; -- T1
update Balls set Color="Black" where Color="White"; -- T2
```

Serial execution will produce either balls that are all white or black. An execution under SI in which the two transactions start from the same snapshot will just swap colors. This anomaly is called **write skew**.

Timestamps in distributed systems A timestamp is an indication of the current time. However, no global time exists. We need some form of **synchronization**.

We assume that we have a system's function that gives out timestamps formatted as **`eventId.nodeId` where `eventId` is unique for each node**. We synchronize different nodes by sending/receiving messages and **imposing that for a given message `send()` precedes `receive()`**. This means that **if for some reason I receive a message from the future, I increase my timestamp such that the `receive` is greater that the corresponding `send` (Lamport method)**.

Reliability control

Reliability control **ensures that transactions are atomic and durable**. The reliability manager **realizes `commits` and `aborts`, orchestrates IO to pages and handles recovery after failures**.

Durability and stable memory

Durability implies a **memory whose content lasts forever**. This means that we cannot use conventional storage as:

- **Main memory** is not stable since it is **not persistent**.
- **Mass memory** is not stable since **it is persistent, but can be damaged**.

Of course this is an **abstraction**. In reality **stability is achieved by replication** (online (RAID) or offline (backups)) and **write protocols**.

Buffers Stable memory is slow. We **can speed things up** by using:

- **Buffers to cache data**
- **Defer writing onto secondary storage**

In main memory a DBMS stores the **buffered content, organized in pages, plus additional metadata: how many transactions are using the page and a flag indicating if the page has been modified and must be aligned to secondary memory**.

The **primitives** that buffer management works with are:

1. `fix`: **loads a page into the buffer**, returns a reference to the page and increments the usage count
2. `unfix`: the opposite of `fix`; **deallocates** and decrements the usage count
3. `force`: **flushes synchronously** a page from buffer to disk
4. `setDirty`: **flips the dirty bit** of a page
5. `flush`: **flushes asynchronously pages** from buffer to disk **when a page is no longer needed**

The **`fix` primitive follows this algorithm**:

1. **Search** for the page in the buffer, **if present increment** the usage counter and return a reference
2. **Select a free page in the buffer** (FIFO or LRU), if present return reference and increment usage counter. **If the dirty bit is set, flush the previous contents of the page to disk**.
   - **If no page is found**, we select a page to deallocate by one of these two policies:
     1. **Steal**: grab a victim page from an active transaction and flush it to disk.
     2. **No steal**: put the transaction in a waiting list until a page frees.

Other buffer management **policies** include:

1. **Force**: pages are always transferred at commit
2. **No force**: transfer can be delayed by the buffer manager
3. **Pre-fetching**: anticipate loading of pages that are likely to be read
4. **Pre-flushing**: anticipate writing of de-allocated pages

Failure handling

A transaction is an atomic transformation from an initial state into a final state. We have **three possible outcomes**:

1. **Commit**: yee
2. Rollback or other faults before commit: we have to **undo** the transaction
3. If we encounter a fault after the transaction we may have to **redo** it

In **case of faults**, we can have two behaviours:

- If failure occurs **between commit and buffer flush**, to ensure durability the reliability manager has to **redo (roll-forward) all the updates**
- If a transaction **had not committed at failure time**, the reliability manager has to **undo any effect of that transaction to preserve atomicity**

A **transaction log** keeps track of the various transaction happening in the DBMS. It is a **sequential file made of records** describing the actions carried out by the various transactions. The log records on **stable memory** in the form of **state transitions** the actions carried out by the various transactions:

- `UPDATE(U)`: both before and after state are stored
- `INSERT`: only after state is saved
- `DELETE`: only before state is saved

We can use the log to apply the following transformations:

1. `UNDO T`: sets the state to the before state
2. `REDO T`: sets the state to the after state

Both **operations are idempotent**. This is necessary since the manager could undo/redo operations twice.

The log contains **different type of records** depending on the type of operation it records:

- Transactional commands: `B(T), C(T), A(T)`
- `UPDATE`, `INSERT` and `DELETE`: `U(T, O, BS, AS)`, `I(T,O,AS)`, `D(T,O,BS)`
- Recovery actions: `DUMP`, `CKPT(T1,..., TN)`

Where `Ti` is the transaction identifier, `O` is the object identifier and `BS`/`AS` are respectively the before and after state.

The **log management rules** ensure that transactions implement **write operations in a reliable way**:

- A **commit record** is always **written synchronously**
- (**Write-ahead log**) The before part of the record must be written before carrying out the action.
- (**Commit rule**) The after state part of the record must be written in the log before carrying out the commit

Since **database writes are async**, different implementations are possible with different impacts on recovery.

- If we **write** onto the database **before commit**, a **`REDO` is not necessary** since the state of the database reflects that of the log.
- If we **write** onto the database **after commit**, the **`UNDO` is not necessary** and it **doesn't require writing the before-states** of objects on the db in order to **abort**.
- If we **write** onto the database **arbitrarily**, we can **better optimize buffer management** however in the general case **we need both `REDO` and `UNDO`**

We have **different types of failure** depending on the type of memory it fails

1. **Soft failure**: we lose the content of the **main memory**.
   - Requires a **warm restart**. We use the log to replay transactions
2. **Hard failure**: we lose part of the **secondary memory**.
   - Requires a **cold restart**. We use a dump to restore the database and the log to replay transactions.
3. **Disaster**: we lose **stable memory**. Not discussed.

Checkpoints Periodically, the reliability manager identifies **consistent time points**:

- All **transactions that committed are flushed to disk**
- All **active transaction are recorded in the log**

The aim is to **record which transactions are still active** at a given point in time.

We have different methods for implementing a checkpoint. A general one is as follows:

- Acceptance of **commit/abort is suspended**
- All **dirty pages** modified by committed transactions are **`forced` to disk**

- The **identifiers of the transactions still in progress are recorded** in the `CKPT` record in the log. **While this record is being made no transaction can start**.

Dumps Dumps are simpler than checkpoints, and are a **complete backup copy of the database**. The **availability** of that copy is **recorded** in the log. The contents of the dump are stored in stable memory.

Warm restart It is a warm restart and consists in a loss of memory buffer pages. It requires replaying transactional operations and resolving eventual problematic situations. Log records are **read starting from the last checkpoint** and **transactions are divided into two sets**:

1. `UNDO` set: transactions to be undone. This set consists of **transactions that were active at checkpoints** plus those that have been **started but not finished**.
2. `REDO` set: transactions to be redone. This set consists of **transactions that have been committed**.

The algorithm is as follows:

1. Starting from `HEAD` **find the last checkpoint**
2. **Construct the two sets** reading from checkpoint to `HEAD`
3. **Return to the first operation of the oldest active transaction while** `UNDO`**ing**
4. **Execute** `REDO` actions until the top of log.

Cold restart It is a loss of secondary memory devices. Data needs to be **restored starting from the last dump**. The **operations recorded** onto the log until failure are **executed**. Then a **warm restart is executed**.

Triggers

The fundamentals have been already discussed in the DB1 course.

A trigger can have 2 **execution modes**:

1. **Before**: the action of the trigger is executed before the **database change** (if the condition holds). This type of **triggers cannot update the database directly, but can affect the transition variables in row-level granularity**.
2. **After**: the action of the trigger is **executed after the modification** of the database.

We have also two different **granularity modes**:

1. **Row-level** (`for each row`): The trigger is considered and possibly executed **once for each tuple affected by the activating statement**.
2. **Statement-level** (`for each statement`): The trigger is considered and possibly executed **once for each activating statement, independently of the number of affected tuples** in the target table.

Special variables denoting the before and after state of the modification are **made available in trigger definition**:

- For `for each row` trigger we have `old` and `new` representing the tuple values respectively before and after modification
- For `for each statement` we have `old table` and `new table` working similarly as seen with row-level granularity

Variables `old` and `old table` are undefined for triggers activated by `INSERT`. Dually, `new` and `new table` are unavailable for `DELETE`.

In SQL99, if multiple events are associated with the same event, an **execution sequence is prescribed**:

1. `BEFORE` statement level triggers
2. `BEFORE` row level triggers
3. Modification is applied and integrity is checked
4. `AFTER` row level triggers
5. `AFTER` statement level triggers

If there are **several triggers in the same category**, the execution order is **system dependant** (based on definition time or alphabetical order).

Cascade and recursive cascading

The action of a trigger can cause another trigger to fire. We say that we are:

1. **Cascading:** when the action of `T1` **triggers** `T2`
2. **Recursive cascading**: when a **statement** `S` **on table** `T` **start a cascade of triggers that generates the same event** `S` on `T`

To guarantee the proper functionality of our database, we need to enforce some rules.

Termination

Termination: for **any initial state** and **any sequence of modification**, a **final state is always produced**.

The simplest check exploits the **triggering graph**:

- A node i for each trigger
- An arc from a node to another if the execution of the trigger associated with the first node may activate the second one

The graph is build with a simple syntactic analysis. If the graph is **acyclic**, the system is **guaranteed to terminate**. If a **cycle exists**, **triggers may not terminate (acyclicity is sufficient for termination)**.

Most systems set a maximum of 64 cascades.

Materialized views

When a view is mentioned in a `SELECT` query, the processor rewrites the query language using the view definition, so that the actually executed query only uses the base tables. When **queries to a view are more frequent than the updates on the base tables that change the view content**, then **view materialization** can be used. Materialization consists in **storing the results of the query that defines the view in separate table**.

Some DBMSs support the `CREATE MATERIALIZED VIEW` **command** that does this automatically. **As an alternative** we can implement **the materialization by means of triggers**.

Design principles

- Use trigger to guarantee that when a specific operation is performed, related actions are performed.
- Do not use triggers to duplicate features of the DBMS.
- Keep triggers small.
- Use triggers only for global, centralized operations.
- Avoid recursive triggers, unless absolutely necessary.
- Use them with parsimony, as they are executed every user interaction.

Ranking queries

The field that studies **optimization based on different criteria** is called **multi-objective optimization**. The general formulation is: **given N objects described by d attributes, and some notion of "goodness" of an object, find the k best objects**.

We have **2 main approaches**: **ranking** (top k objects according to a score), **skyline** (set of non-dominated objects).

We will use a **metric approach to ranking**: we find a **ranking whose total distance to the initial rankings is minimized**. Several **distances** between ranking are defined e.g.:

1. **Kendall tau**: the number of exchanges in a bubble sort to convert R_1 to R_2
2. **Spearman's footrule**: adds up the distance between the ranks of the same item in the two rankings

Finding an **exact solutions for Kendall-tau is NP-complete**, while finding a **solutions for Spearman is polynomial**. In addition to being more efficient, Spearman admits efficient approximations (e.g. median ranking)

Combining opaque rankings

Uses only the position of the elements in the ranking, no other associated scores.

MedRank

It is based on the notion of median and provides an **approximation of Spearman's optimal aggregation**.

- **Input**: k, ranked lists $R_1 \dots R_m$ of N elements
- **Output**: the top k elements according to median ranking:

  Use sorted access in each list, one element at a time, until there are k elements that occur in more than $m/2$ lists. There are the to k elements.

MedRank is not optimal, however it is instance optimal: among the algorithms that accesses the lists in sorted order, this is the best possible algorithm on every input instance.

Optimality vs instance optimality An algorithm is said to be optimal if its execution cost is never worse than any other algorithm on any input.

Instance optimality is a form of optimality aimed at when standard optimality is unachievable: algorithm $A^\star$ is instance-optimal w.r.t a family A and I problem instances for the cost metric c if there exists k_1, k_2 such that

$$\forall A' \in A, I' \in I \, c(A^\star, I') \leq k_1 \cdot c(A', I') + k_2$$

This means that if $A^\star$ is instance-optimal, then any algorithm can improve the cost by only a constant factor r called the optimality ratio of $A^\star$.

Instance optimality is a much stronger condition than optimality in the average or worst cases.

Top-K queries

The aim is to **retrieve only the k best answers from a potentially large result set**. We need an ability to rank objects based on a metric.

Naive approach

Assume a scoring **function S that assigns to each tuple t a numerical score for ranking tuples**. We use the most straightforward version for computing the result.

- **Input**: cardinality k, dataset R, scoring function S
- **Output**: the k highest-scored tuples w.r.t S
  1. For all tuples t in R: compute $S(t)$
  2. Sort tuples based on their scores
  3. Return the first k highest scored tuples

This approach is very expensive for large datasets. Even worse, if more than one relation is involved we need to join all tuples.

SQL

We need two abilities: **ordering and limiting**. Ordering is done by `order by`, while limiting is done by `fetch first k rows only` (not in standard until 2008, each DBMS has its own way).

Calculating **order is done by assigning weights to various metrics to normalize values**.

```sql
-- 1
SELECT * FROM UsedCars WHERE Vehicle = 'Audi/A4' and Price <= 21000
ORDER BY 0.8*Price + 0.2*Miles

-- 2
SELECT * FROM UsedCars WHERE Vehicle = 'Audi/A4'
ORDER BY 0.8*Price + 0.2*Miles
```

`order by` is not perfect. It can be subject to 2 problems:

1. **Near-miss**: some relevant information is lost (first query)
2. **Information overload**. (second query)

Evaluation

We have many scenarios possible. Let us consider two aspects to consider: query type and access paths. The simplest case is a top-k selection with only 1 relation:

- If input is sorted according to S: we read only the first k tuples
- If tuples are not sorted, if k is not too large, we can perform a in-memory sort ($\mathcal{O}(n \log(k))$ cost)

K-Nearest neighbour

Let us consider **distances** rather than scores. The model is now:

- A **m-dimensional space** of ranking attributes $A = (A_1, \ldots, A_n)$
- A **relation** $R(A_1, \ldots, A_n, B_1, \ldots, B_m)$ where B_x are other attributes
- A **target point** $q \in A$
- A **function** $d : A \times A \to \mathbb{R}$ measuring the distance between two points of the attribute space

Under such model a **top-k query is transformed into a so called k-nearest neighbours query**: given a point q, a relation R, $k \geq 1$ and a distance d, determine the k tuples un R that are closest to q according to d.

Some commonly used distances are **Lp-norms**:

$$L_p(t, q) = (\sum_{i=1}^{m} |t_i - q_i|^p)^{1/p}$$

Some relevant special cases are:

1. **Euclidean**: $L_2(t, q) = \sqrt{\sum_{i=1}^{m} |t_i - q_i|^2}$
2. **Manhattan**: $L_1(t, q) = \sum_{i=1}^{m} |t_i - q_i|$
3. **Chebyshev**: $L_\infty(t, q) = \max_i\{|t_i - q_i|\}$

The use of weights stretches some coordinates (e.g $L_2(t, q, W) = \sqrt{\sum_{i=1}^{m} w_i |t_i - q_i|^2}$.

Top-K join query

In a top-k join query we have $n > 1$ input relations and a scoring function S defined on the result of the join:

```sql
SELECT        A1, A2, ...
FROM          R1, R2, ...
WHERE         ...
ORDER BY      S(p1, p2, ...) [DESC]
FETCH FIRST   k ROWS ONLY
```

`p1, p2, ...` are scoring criteria, or preferences.

All the joins are on a common key attribute. This is the simplest case to deal with and is the basis for more general cases.

Top-k join queries are **used in mainly two scenarios**:

- **There is an index for retrieving tuples according to each preference**.
- The relation is spread over several sites, each providing information only on part of the objects (the **"meta-search-engine" scenario**).

Each of this scenarios **assumes** that:

1. Each input list **supports sorted access** (each access returns the id of the next best object)
2. Each input list **supports random access** (each access returns the partial score of an object given its id)
3. The **id of an object is the same across all inputs**
4. Each **input consists of the same set of objects**

Each object o returned by L_j has an **associated partial score** $p_j(o) \in [0;1]$ **The hypercube** $[0;1]^m$ **is called the score space. The point** $p(o)$ **is the map of object** o **into the score space.** The **global score** $S(o)$ is **computed by means of a scoring function that combines in some way the local scores of** o. Some common scoring functions are:

1. Sum
2. Weighted sum
3. Minimum
4. Maximum

We define iso-score curves in the score space as a set of points that have the same global score.

Scoring function: `MAX` We use the B_0 **algorithm**.

- **Input**: integer $k \geq 1$, ranked lists $R_1, \ldots, R_m$
- **Output**: the top-k objects according to the max scoring function
  1. Make k sorted accesses on each list and store objects and partial scores in a buffer B
  2. For each object B, compute the max of its available partial scores
  3. Return the k objects with maximum score

We **do not need to obtain missing partial scores** and we **do not execute any random access**.

Fagin's algorithm **Works with any monotone function**.

- **Input**: integer $k \geq 1$, a monotone function S combining ranked lists $R_1, \ldots, R_m$
- **Output**: the top-k `<object,score>` pairs
  1. Extract the same number of objects by sorted accesses in each list until there are at least k objects in common
  2. For each extracted object, compute its overall score by making random accesses wherever needed
  3. Among these, output the k objects with the best overall score

Complexity **is sublinear in the number** N **of objects**: $\mathcal{O}(N^{(m-1)/m}k^{1/m})$.

The drawback of this approach is that the **scoring function is not exploited**. Plus **memory requirements can become prohibitive**.

Threshold algorithm

- **Input**: integer $k \geq 1$, a monotone function S combining ranked lists $R_1, \ldots, R_m$
- **Output**: the top-k `<object,score>` pairs
  1. Do sorted accesses in parallel in each list R_i
  2. For each object o, do random accesses in the other lists R_j, thus extracting score s_j.
  3. Compute the overall score $S(s_1, \ldots, s_m)$. If the value is among the k highest seen so far, remember o
  4. Let s_{Li} be the last score seen under sorted access for R_i
  5. Define threshold $T = S(s_{L1}, \ldots, s_{Lm})$
  6. If the score of the k-th object is worse than T, go to (1)
  7. Return the current top-k objects

TA is instance optimal among all algorithms that use **random and sorted accesses**. An improvement over FA is that the **stopping criterion depends on the scoring function**.

In general, TA performs much better than FA, since it can adapt to the specific scoring function. **In order to characterize the performance of TA, we consider the middleware-cost:**

$$c = SA \cdot c_{SA} + RA \cdot c_{RA}$$

Where:

- SA (RA) is the **total number of sorted (random) accesses**
- c_{SA} (c_{RA}) is the **unitary (base) cost of a sorted (random) access**

In a basic setting, $c_{SA} = c_{RA} = 1$. In other cases, **costs may differ**:

- For web sources, usually $c_{RA} > c_{SA}$ with the limit case where $c_{RA} = \infty$ (random access is impossible)
- Some sources might not be accessible through sorted access ($c_{SA} = \infty$)

NRA algorithm No Random Access (NRA) **applies when random accesses cannot be executed**.

It **returns the top-k objects, but their scores might be uncertain**. The idea is to maintain for each object o retrieved by sorted access lower and upper bounds $S^-(o), S^+(o)$ on its score. NRA uses a buffer with unlimited capacity, which is kept sorted according to decreasing lower bound values.

Algorithm:

1. Make a sorted access to each list
2. Store in B each retrieved object o and maintain the bounds a threshold τ
3. Repeat step 1 as long as

$$S^-(B[k]) < \max\{\max\{S^+(B[i]), i > k\}, S(\tau)\}$$

NRA is instance optimal. Its **cost does not grow monotonically with** k: i.e it **might be cheaper to look for the top-**k **objects rather than for the top-**$(k-1)$.

Skyline queries

Skyline queries work **based on the concept of dominance** between different objects: a **tuple** t **dominates** s ($t \prec s$) iff:

- $\forall 1 \leq i \leq m \, t[A_i] \leq s[A_i]$
- $\exists 1 \leq j \leq m : t[A_i] < s[A_j]$

Typically lower values are considered better. The **skyline of a relation is the set of its non-dominated tuples**.

A tuple t **is said to be in the skyline iff it is the top-1 result w.r.t at least one monotone scoring function** (i.e the skyline is the set of potentially optimal tuples).

SQL

Only a proposed syntax is available:

```
SELECT ... FROM ... WHERE ...
GROUP BY ... HAVING ...
SKYLINE OF [DISTINCT] d1 [MIN | MAX | DIFF],
                       ...
ORDER BY ...
```

Block Nested Loop

We can implement a skyline **naively** using **nested queries**:

```
SELECT * FROM Hotels h WHERE city = 'Paris' AND NOT EXISTS (
  SELECT * FROM Hotels h1
  WHERE h1.city = h.city and
    h1.distance <= h.distance and
    ...)
```

However this query is **very slow (quadratic complexity)**.

The algorithm is schematized as follows:

```
def bnl(D):
  W <- empty_set
  for each p in D do:
    if (p is not dominated by any point in W) then:
      W <- W - p.dominated_points
      W <- W + {p}
  return W
```

Sort-Filter-Skyline

We improve BNL by **pre-sorting**: if the input is sorted, later tuples cannot dominate any previous tuple. **However, complexity remains still quadratic**.

```
def sfs(D, sort_fn):
  S <- sort(D, sort_fn)
  W <- empty_set
  for each p in D do:
    if (p is not dominated by any point in W) then:
      W <- W + {p}
  return W
```

JPA

Working directly with JDBC can lead to a lot of boilerplate code. JPA solves the repetitiveness of interacting directly with JDBC by creating some **useful abstractions**.

Object relational mapping

Defines a **mapping between SQL entities and Java objects**. The main way to do it is using **annotations**. The problem of **"impedance mismatch"** occurs when **one model cannot be cleanly mapped onto another one**. A **mediator** (JPA in this case), is **needed to manage the transformation of one into the other**.

Object model	Relational model
Classes, objects	Tables, rows
Attributes, properties	Columns
Identity	Primary key
Refrences	Foreign key
Inheritance/Polymorphism	NA
Methods	Stored procedures, triggers

JPA works with POJOs in a non-intrusive way. A query framework is provided instead of raw SQL.

Main concepts:

1. **Entity**: a Java **bean** representing a collection of **persistent objects mapped onto a relational table**
2. **Persistence unit**: the set of **all classes that are persistently mapped to one database**
3. **Persistence context**: the set of **all managed objects of the entities defined in the persistence unit**
4. **Managed entity**: an **entity part of persistence context** for which the **changes of the state are tracked**
5. **Entity manager**: the **interface for interacting with a persistence context**
6. **Client**: a component that can interact with a persistence context indirectly through an entity manager

We will write the client and interact only with the entity manager.

Entities

An entity is a **POJO** that gets **associated to a tuple in a database**. The persistent counterpart of an entity has a life longer than that of the application. The **entity class must have a mapping to the table it represents**. An entity **can enter in a managed state**, where **all modifications to the state are tracked** and automatically written to the database.

```
@Entity
public class Employee {
  @Id private int id; // defines the primary key
  private String name;
  private long salary;
  // constructors, getter and setter boilerplate
}
```

Entities must be `public/protected` and not `final`; no method or persistent instance variables may be final; **if an instance is to be passed by value as a detached object, `Serializable` must be implemented**.

Primary keys are identified by `@Id`, if they are **complex objects** we use **`@EmbeddedId` and `@IdClass`**. Identifiers **can be generated automatically** by the framework using `@GeneratedValue` in addition to `@Id` with **different strategies**:

1. `AUTO`: the framework decides
2. `TABLE`: generated according to a generato table
3. `SEQUENCE`: if the underlying db supports sequences, it uses that
4. `IDENTITY`: if the underlying db supports primary key identity columns, the provide will use that

Properties can be **annotated with type information** to better map Java types to db types (e.g `@Temporal()` or `@LOB`).

We can also specify **fetch policy** (lazy ore eager). The **lazy** policy makes the **attribute values remain empty when the object is retrieved and populated only when said property is accessed**.

By default, **entities are mapped to tables with the same name and their fields to columns with the same names**. We can **override** said defaults with: `@Table(name="")` and `@Column(name="")`. Some annotations have also attributes for generating the database schema from the entity class (e.g `nullable`). Entities can also have **attributes that are not persisted** (`@Transient`).

Relationships Relationships in the OM have 4 characteristics:

1. **Directionality**: each entity has a reference to another entity (unidirectional) or both entities have references to each other (bidirectional).
2. **Role**: Based on directionality, an entity can be either the source or the target.
3. **Cardinality**
4. **Ownership**: In the database relationships are implemented by a foreign key column that refers to the key of the referenced table (join column). The entity that will have FK is the oner of the relationship.

An example of a bidirectional 1:N.

```
@Entity
public class Emplyee {
```

```
  @Id private int id;
  @ManyToOne
  @JoinColumn(name="dept_fk")
  private Department dept;
  // ...
}

@Entity
public class Department {
  @Id private int id;
  @OneToMany(mappedBy="dept") // mappedBy takes the name of the parameter that
                              // holds the reference.
  private Collection<Employee> employees;
  // ...
}
```

The purpose of the `mappedBy` is to **instruct JPA to not use a bridge table like in a N:M relationship as the relationship is already being mapped by a FK in the opposite entity of the relationship**. Generally, `mappedBy` **indicates to JPA that the current entity is the owned side** of the relationship and that the owner FK resides in the indicated property of the owner class. `mappedBy` is used to specify **bidirectional relationships**.

Example of 1:1 relationship. The owner can be both entities. We decide.

```
@Entity
public class Employee {
  @Id private int id;
  @OneToOne // owner
  private ParkingSpace parkingSpace
  // ...
}

@Entity
public class ParkingSpace {
  @Id ptivate int id;
  @OneToOne(mappedBy="parkingSpace") // owned. mappedBy necessary only if the
                                     // relationship is bidirectional
  private Employee employee;
  // ...
}
```

Example of N:M relationship. Like in the 1:1 case, we can use either entity as the owner. Many-to-many mappings are implemented by means of a join table.

```
@Entity
public class Employee {
  @Id private int id;
  @ManyToMany // owner
  private Collection<Project> projects;
  // ...
}

@Entity
public class Project {
  @id private int id;
  @ManyToMay(mappedBy="projects")
  private Collection<Employee> employees;
  // ...
}
```

The **name and column names of the join table** can be specified with the **`@JoinTable` annotation**.

```
@Entity
public class Employee {
  @Id private int id;
  @ManyToMany
  @JoinTable(
    name="EMP_PROJ",
    joinColumns=@JoinColumn(name="EMP_ID"),
    inverseJoinColumns=@JoinColumn(name="PROJ_ID"))
  private Collection<Project> projects;
  // ...
}
```

When using **relationships**, we can specify the **fetch mode**. By **default**, a **single valued relationship is fetched eagerly**, while a **collection-valued relationship is lazy**. In case of **bidirectional relationships**, **one side can be lazy while the other eager**. The **lazy** fetch mode is considered more of an **hint**, while the **eager** one is **always respected**.

By **default**, every **operation applies only to the entity supplied as an argument to the operation**. The **`cascade` attribute of relationship attributes is used to define when operations should be automatically cascaded across relationships**. **Cascade** settings are **unidirectional**.

For `**OneToOne` and `OneToMany**` JPA supports an **additional cascade mode**: `orphanRemoval`. It is **appropriate for parent-child relationships** (e.g weak entities). It causes the **child entity to be removed when the relation is broken either by**:

1. **Removing the parent**
2. Setting to **`null` the attribute that holds the related entity**
3. In the `**OneToMany` case**: by** removing the child from the parent's collection**.

Note: normal cascading is not invoked for `setParent(null)`, unlike `orpahnRemoval`.

Adding an **attribute to a relation** can be done by adding some annotations on the owner's side:

```
@Entity
public class Order implements Serializable {
  @Id @GeneratedValue(stategy = GenerationType.AUTO)
  private int OrderId;
  private int shippingFee;
  private int totalCost;

  @ManyToOne
  @JoinColumn(name="customer")
  private Customer  customer;

  @ElementCollection(fetch = FetchType.EAGER)
  @CollectionTable(name = "product_order",
                   join_columns = @JoinColumn(name = "orderId")) // tells JPA that we are mapping to a
  @MapKeyJoinColumn(name = "ProductId") // this is the map key
  @Column(name = "quantity")
  private Map<Product, Integer> products;

  //...
```

We can also we an auxiliary weak entity with a composite key.

Entity manager

1. `void persist(Object entity)`: persists an entity instance in the database
2. `<T> T find(Class<T> entityClass, Object primaryKey)`: finds an entity instance by its primary key

3. `void remove(Object entity)`: Removes an entity instance
4. `void refresh(Object entity)`: Synchronizes the entity instance with the database
5. `void flush()`: Writes the state immediately

The entities only **become managed after an entity manager method is invoked**, otherwise they just plain Java objects.

When a new entity object is **first instantiated**, it is in the **transient state**, since the Entity manager does not know it exists yet. An entity **becomes managed** when **`persist()` is first called**. **Persist can be called on an already managed entity** and it triggers the whole cascade process.

`flush()` instructs the persistence manager to write changes as soon as possible (as will be said in the next section, the persistence manager decides when to write to the database).

`remove()` breaks the association between the entity and the persistence context. When the transaction associated with the entity manager's persistence context commits or the `flush()` method is called, **the tuple associated with the entity is scheduled for deletion**. The entity **exits the managed** state and re-becomes transient.

Entity states

1. **New**: basically transient
2. **Managed**: the state is tracked by the Entity manager
3. **Detached**: Has an identity potentially associated with a database tuple but changes are not automatically propagated to database
4. **Removed**: scheduled for deletion from the database
5. **Deleted**: deleted from the database

Persistence context

It is the fundamental concept of JPA: it is a kind of **main memory database** that **holds the objects in the managed state**. A managed object is tracked, i.e all the modifications to its state are monitored for automatic alignment. This **binding exists only inside the persistence context**, once the object exits it, it returns to being a normal Java object.

Database **writes are asynchronous and occur at points decided by the context**. For the writes to happen, the **Persistence context mush hook up to a transaction**.

Transaction management

In JPA we have different types of transactions:

- **DBMS transaction**: low level SQL transaction
- **Resource-local transaction**: created by JDBC commands
- **Container**: defined by the Java Transaction Manager (JTA) interface and mapped to JDBC. They can be managed by the application of by the container (EJB, servlets or whatever)

In **container-level transactions**, the **entity manager is itself managed** by the container.

```
@Stateless // annotation indicating that this is a EJB object
public class EJBService {
  @PersistenceContext(unitName = "MyPersistenceUnit")
  private EntityManager em;
  // ...
```

When using container managed transactions, **a new transaction is created when a business method that requires it is called and there is not yet an active transaction**. The transaction is **committed** when the **method that caused its creation terminates**.

Transaction propagation is the process of **sharing a persistence context between multiple container managed entity managers**. Thanks to propagation, `methdodA()` of `ComponentA` can call `methdoB()` to `ComponentB` and share the same transaction. **Propagation can be**:

1. **"Vertical"** along the call stack: if `methodA()` calls `methodB()` by default the latter executes in the same transaction as the former. Both the transaction and persistence context are shared.
2. **"Horizontal"**: if the EMs of the two components are defined on the same Persistence unit, the calls to the methods of the different components share the same persistence unit.

How the transaction is shared during method calls is defined by `@TransactionAttribute(TransactionAttributeType.*)` annotation:

- `MANDATORY`: a transaction is expected to have already been started and be active when the method is called. If none is active, an exception is thrown.
- `REQUIRED`: the **default**
- `REQUIRES_NEW`: the method always creates a new transaction, if one already exists it is suspended
- `SUPPORTS`: the method does not access transactional resources, but tolerates running one if it exists
- `NOT_SUPPORTED`: the method will cause the container to suspend the current transaction if one is active
- `NEVER`: the method will cause the container to throw an exception if a transaction is active

Thread safety

The **injected EMs are thread safe** thanks to a series of methods (proxying and method call serialization).

Physical database

Databases are stored onto secondary memory into files. When we want to use data, we **need to move the data to primary memory**. **Data is organized into pages and blocks**:

1. **Page**: unit of storage in **main memory**
2. **Block**: unit of storage in **secondary memory**

Secondary memory devices are **organized in blocks of usually fixed length**. We will make the **assumption that the size of a page equals the size of a block**. We can **estimate the cost of a query with the number of blocks to be moved** to execute said query.

The DBMS uses only the basic features of the file system (read/write, file creation/deletion). The **DBMS directly manages file organization both in terms of distribution of records within blocks and with respect to the internal structure of each block**.

Access structures

Each **access method** (access method: module that provides data access and manipulation primitives for each physical data structure) has **its own data structure**. All **structures are stored into the database**.

Structures are classified into:

1. **Primary structures**: they contain the **tuples** of a table
2. **Secondary structures**: they are used to **index primary structures**, and only contain the values of some fields, interleaved with pointers.

Mainly we have sequential, hash-based and tree-based structures. Not all structures are equally suited for the same job. **Sequential structures are usually used for primary structures** while **hash-based and tree-based structures are used for secondary structures**.

Tuples and blocks

Tuples are the **logical components of tables** while **blocks are the physical components of files**. **Blocks** typically have **fixed size** and **depend on the file system and how the disk is formatted**. **Tuple**, otoh, have **variable size** (dependant on the database design).

Blocks contain tuples. The block is organized as **two juxtaposed stacks with some headers and footers**:

1. **Block header and trailer** contain **control information** used by the filesystem
2. **Page header and trailer** contain **control information** about the access method

3. A **page dictionary** is the first stack and **contains pointers to each item contained in the page**.
4. The **useful part of the page** is the other stack and **contains tuples**
5. A **checksum** is present to spot corrupted data.

The **number of tuples** in a block is given by the **block factor**. It is a fundamental measure for evaluating the cost of queries.

$$B = \lfloor \frac{SB}{SR} \rfloor$$

Where SR is the average size of a record/tuple and SB is the size of the block.

The operation we will consider are: insertion and update of a tuple, deletion of a tuple, access to a field of a particular tuple.

Sequential structures

Tuples are sequentially arranged into blocks. We can have two cases:

1. **Entry-sequenced**: the sequence is dictated by their **order of entry**
   - **Optimal** for **insertion, sequential reading and space**
   - **Non optimal** for **searching and updates** that increase the size of a tuple
2. **Sequentially-ordered**: tuples are **ordered according to the value of a key**
   - **Optimal for range queries and `order by`/`group by` queries** exploiting the key
   - The **problem** is **insertion and updates that increase the size**: they trigger a **reordering**

Hash-based structures

Basically hash-tables, review API.

Indexes

They are data structures that **help efficiently retrieve tuples on the basis of a search key**. They contain records of the form `[key, pointer]`. Index entries are sorted w.r.t the search key.

Indexes can be:

- **Dense**: there is **an entry for each search key value in the file**. We will always assume this type of index.
- **Sparse**: there is an **entry only for some search key values**.
- **Primary**: an index **defined on sequentially ordered structures**. The **search key is unique and coincides with the attribute according to which the structure is ordered (called ordering key)**. Only **one primary index can be defined for table**, usually on the primary key.
- **Clustering**: a **generalization of the primary index** where the **ordering key can be not unique**. The pointer will simply **refer to the block containing the first tuple matching the key**.
- **Secondary**: an index that **specifies an order different from the sequential order of the file**. **Multiple secondary indexes can be defined on the same tables for different search keys**. It is **necessarily dense**.

Indexes are **usually implemented as hash-based structures**. They are handled exactly like a hash-based primary structure but instead of tuples the buckets only contains key values and pointers.

To **access an index**, we need to **access two blocks**: one with index and one with the data.

Tree-based structures

The **most frequent data structures for secondary (index) structures**. We have two types trees: **B trees and B+ trees**. We will see B+ trees.

Each node of the tree is stored in a block. Internal nodes form a multilevel sparse index on the leaf nodes. Each **node** can hold **up to** $N-1$ **search keys and** N **pointers**. **At least** $\lceil N/2 \rceil$ **pointers are**

maintained to ensure that **each internal node is at least half full**. **Search keys** in a node are **sorted with** $K_i < K_j$.

```
[ P_1 | K_1 | P_2 | K_2 | ... | P_i | K_i | P_i+1 | ... | K_N-1 | P_N ]
   |                          |                                  |-> subtree with key K >= K_N+1
   |                          |-> subtree with keys K_i-1 <= K <= K_1
   |-> subtree with keys K <= K_1
```

Each **leaf node** can **hold up to** $N-1$ **search keys**. **At least** $\lceil (N-1)/2$ **key values should be present in each node**. Search **keys are sorted in the same way as internal nodes**. The **last pointer in the leaf node is a pointer to the next leaf** to allow sequential processing. The set of leaves **forms a dense index**.

```
[ P_1 | K_1 | P_2 | K_2 | ... | P_i | K_i | P_i+1 | ... | K_N-1 | P_N ]
   |                          |                                  |-> pointer to next leaf
   |                          |-> Pointers to blocks of primary stoage containing key K_i
   |-> Pointers to blocks of primary storage containing tuples with keys K_1
```

B trees eliminate the redundant storage of search key values: search key values appear only once; intermediate nodes for each key value K_i have:

- One pointer to the subtree with keys between K_i and K_{i+1}
- Pointers to the blocks that contain the tuples that have value K_i for the key

Lookup can be slightly faster, but interval queries are less efficient.

Indexes in SQL

Indexes are created with `create [unique] index IndexName on TableName(AttrList)` and dropped with `drop index IndexName`.

In most SQL DBMSs, every table should have a suitable primary storage, possibly sequentially ordered. Secondary structures are progressively added, checking that the system actually uses them.

Some **guideline for choosing indexes**:

1. Do not index small tables
2. Index primary keys of a table if it not a key of the primary file organization (some DBMSs automatically generate indexes for primary keys)
3. Add a secondary index to any columns that is used heavily as a secondary key
4. Add a secondary index to a foreign key if it is frequently used
5. Add secondary indexes on columns that are involved in:
   - selection or join criteria
   - `order by`
   - `group by`
   - other operators involving sorting (e.g `union` or `distinct`)
6. Avoid indexing a columns or table that is frequently updated
7. Avoid indexing a column if the query will retrieve a significant portion of the records
8. Avoid indexing columns that consist of long strings

Query optimization

Costs of different queries

The **query optimizer** receives a **SQL query and produces a program in an internal format that uses the data access method**. The fundamental steps in the translation process are the **algebraic optimization and the cost-based optimization**.

Profiles are stored in the data dictionary and contain **quantitative information about tables** like:

1. **Cardinality**
2. **Dimension** in bytes of each **attribute**
3. **Number of distinct values** of each **attribute** ($val(A_j)$)

4. **Minimum/maximum values** of each attribute.

These statistics are **used by the query optimizer for cost-based optimizations** and are periodically calculated.

Selectivity: the **probability that any row will satisfy a predicate**. If $val(A) = N$ and the **values are homogeneously distributed over the tuples** then it can be calculated as $1/N$ **for predicates in the form of** $A = k$.

Operations supported by various access modes

A sequential structures perform a **sequential access** to all the tuples of a table/intermediate result executing various operations such as projection or selection on simple predicates.

Hashing adds support to equality predicates. Tree-based indexes **support selection/join criteria,** `order by`**,** `group by` **and other operations involving sorting**.

Costs of lookups for different primary/secondary structures

Predicate-based lookup can use indexes built un the attribute for queries with equality or interval functions. For equality lookup we have:

1. **Sequential**:
   - **Entry sequenced**: we need a **full scan**
   - **Sequence ordered**: if we cannot **exploit our** k **we can reduce the cost** or still need a **full scan**
2. **Hash**:
   1. **Primary**:
      - If we have **no overflow chains** we have just **1 access**, otherwise we need to **follow the chain with an extra cost**.
   2. **Secondary**:
      - Like in the **primary case**, we have the cost of 1 + a scan of the overflow chain if preset. Since it is a secondary index, we also need to **add the cost of accessing the full tuples in primary storage**.
3. **B+ tree**:
   1. **Primary**:
      - **Unique search key**: we need to access k **intermediate plus 1 leaf node**
      - **Non search key**: we need k **intermediate plus a full scan of all the leafs**
      - **Non-unique search key**: we need k **intermediate plus** j **leafs**. Since the search key is non-unique, it can appear in multiple leafs. This means that **search algorithm needs to be extended**: if the key is the first in a leaf, we also need to visit the previous sibling; the visit continues accessing all the next leaf nodes until a key greater than our K is found.
   2. **Secondary**:
      - **Unique search key**: like the primary case + the access to the data block
      - **Non search key**: 1 full scan for leaves and one for data
      - **Non-unique search key**: like in the primary case + the access to the data blocks

Interval lookups are not supported by sequential and hash structures since they require a full scan. **Tree structure support it if the attribute on which we are making the lookup is the search key**.

1. **Primary**: we consider $v_1 \leq A_i \leq v_2$ as the general case
   - First we read the root. Then **a node per level is read until the leaf node is reached that stores tuples with** $A_i = v_1$. If the searched tuples are all stored in that leaf block we finish, otherwise we **continue up the leaf chain until** v_2 **is reached**.
   - The cost is **1 block per level + as many leaf blocks necessary** to read all tuples in the interval
2. **Secondary**: we consider $v_1 \leq A_i \leq v_2$ as the general case
   - The algorithm is identical to the primary case
   - The cost is identical to the primary case, plus the 1 access to data blocks per pointer.

Conjunction and disjunction of predicates is done as follows:

1. **Conjunction**: If supported by the indexes, we **choose the most selective supported predicate** for data access and **evaluate the other predicates in main memory**
2. **Disjunction**:
   - If all predicates are supported, **indexes can be used for each predicate and then duplicate elimination is performed**
   - If **any of the predicates is not supported by the indexes, then a full scan is needed**

Sorting

We distinguish two type of sorting:

1. **Sorting in main memory**, typically done by means of the usual algorithms
2. **Sorting of large files that do not fit into main memory**, done with special algorithm like external merge sort.

External merge sort

The general idea is to **first sort chunks of data small enough to fit in main memory and them merge the sorted parts**.

To sort a file stored in N blocks using B buffer pages we:

1. **Read B blocks at a time into main memory and sort them**. Then **write the sorted data into a chunk file**. The total chunks will be $\lceil N/B \rceil$
2. We use $B-1$ blocks for the input and 1 block for the output. **Until all input buffer pages are empty**:
   - **Select the first record** (in sort order) among all buffer pages and **write it to the output buffer;** if full, write it to disk.
   - **Delete the record from its input buffer page**. If the buffer page is empty, read next block of the chunk into the buffer.

In each pass, $B-1$ chunks are merged. A pass reduces the number of chinks by a factor of $B-1$ and creates chunks longer by the same factor. We can calculate the **number of passes as** $1 + \lceil \log_{B-1} \lceil N/B \rceil \rceil$. The **overall cost of the sort is** $2N * \#_{passes}$.

Joining

Joins are the **most frequent and costly operations** in DBMSs. We have several join strategies, among which we will see: nested loops, merge-scans and hashed.

Nested loop join

A nested loop join **compares the tuples of a block of table T_{Ext} with all the tuples of all the blocks of T_{Int} before moving to the next block of T_{Ext}.**

We will **assume that the buffer does not have enough available free pages to host more than a few blocks**. The **cost** of the join will be **quadratic in the size of the tables**. If **one of the tables is small enough to fit in the buffer**, then it is **chosen as the internal table**.

If **one table supports indexed access in the form of a lookup based on the join predicate, then this table can be chosen as internal**, exploiting the predicate to extract the joining tuples without scanning the entire table. If **both tables support lookup, then the one for which the predicate is more selective is chosen**. The **cost** will be a **full scan of the external blocks + the cost of one indexed access of the internal table**.

Merge-scan join

Merge-scan is possible only if both tables are ordered according to the same key attribute, that is also the attribute used in the join predicate. **If the tables are not sorted, we first sort them on the join column**.

The **ordered full scan is possible** for a table if the **primary storage is sequentially ordered w.r.t the join attribute or a B+ on the join attribute is defined**.

The **cost is linear** in the size of the tables. If a table **needs to be sorted**, we **need to also add the cost for sorting**.

Hashed joins

This join is possible only is **both tables are hashed according to the same key (join) attribute**. The matching tuples can only be found in the corresponding buckets.

The **cost is linear in the number of blocks of the hash based structure**.

Cost-based optimization

It is a problem whose decisions are: the data access operations to execute, the order of operations, the option to allocate to each operation and deciding if parallelism/pipelining can be used.

Queries are optimized by using profiles and approximation formulas. Using these we **build decision trees** in which each **node is a choice** and each **leaf corresponds to a specific execution plan**.

For **query execution** we have two paths:

1. **Compile and store**: the query is compiled once and executed many times **(prepared statement)**
2. **Compile and go**: the query is immediately compiled and executed.